

SPHERE

Executive Summary

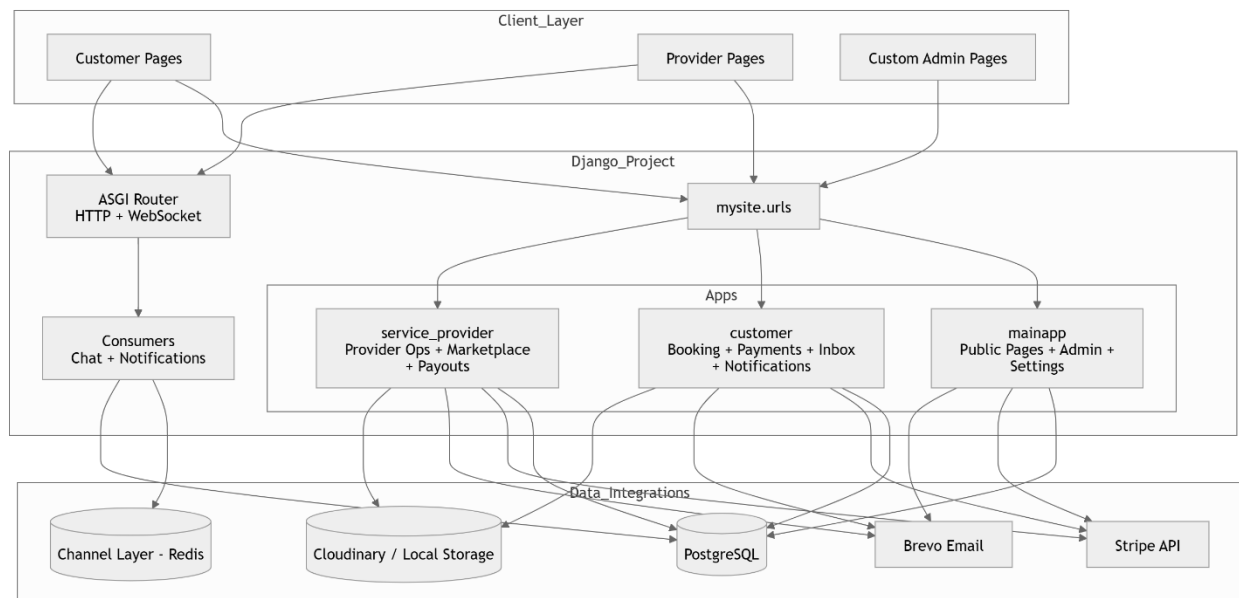
SPHERE is a Django 5 service marketplace platform connecting customers and service providers, with:

- customer-provider booking lifecycle
- Stripe payments and payout tracking
- provider product marketplace
- real-time chat and notifications via Channels/WebSockets
- custom admin dashboard workflows (approvals, payouts, promo, exports)

Core apps:

1. **mainapp**: public pages, blog, custom admin dashboard, platform settings
2. **customer**: customer account, booking/payment, inbox, analytics, notifications
3. **service_provider**: provider account, product marketplace, payouts, portfolio, provider ops

2) High-Level Architecture



- Framework: Django 5.x

- Async stack: Channels + Daphne (ASGI)

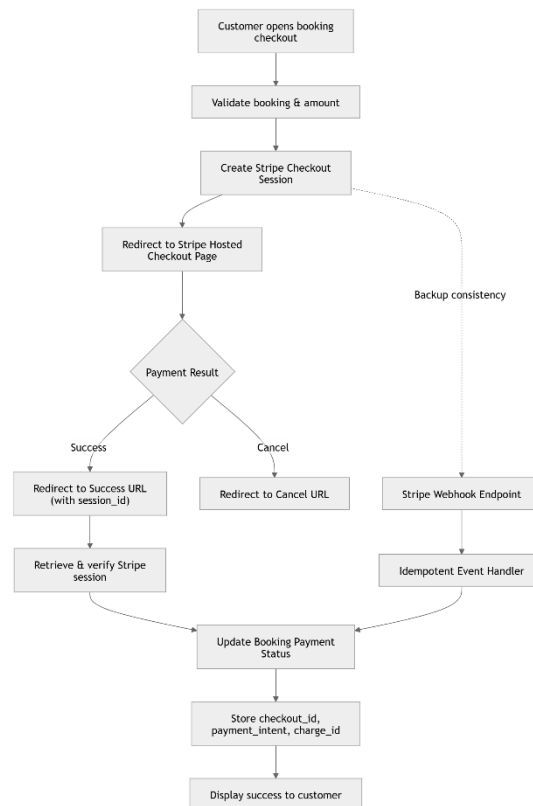
- WebSocket routes: chat and notifications
- Database: PostgreSQL (configured in settings.py)
- File storage: local media or Cloudinary (env-driven)
- Payment gateway: Stripe (keys from env and/or DB singleton settings)
- Email: custom Brevo backend

Runtime entrypoints:

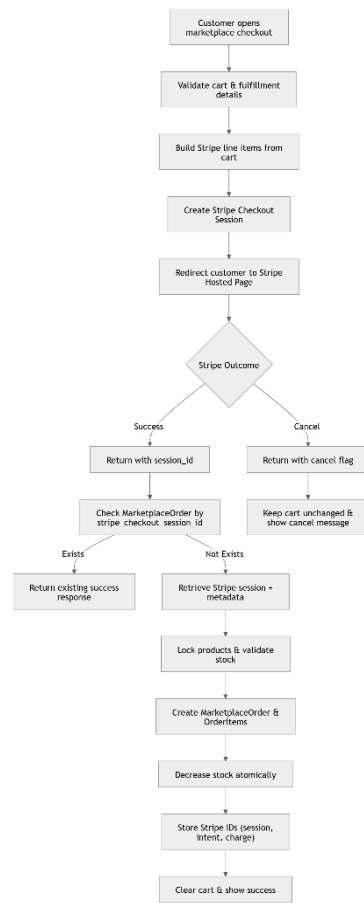
- ASGI app: `mysite/mysite/asgi.py`
- URL root: `mysite/mysite/urls.py`
- Deployment start command uses Daphne with migrate on boot

Flowchart

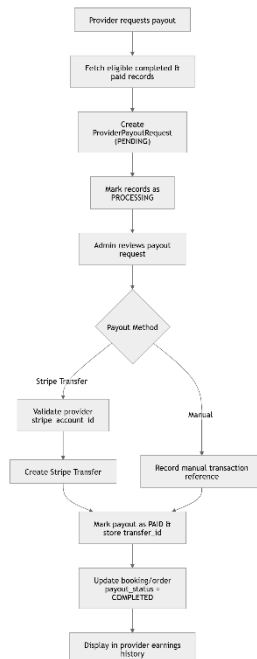
Customer Booking Payment Flow



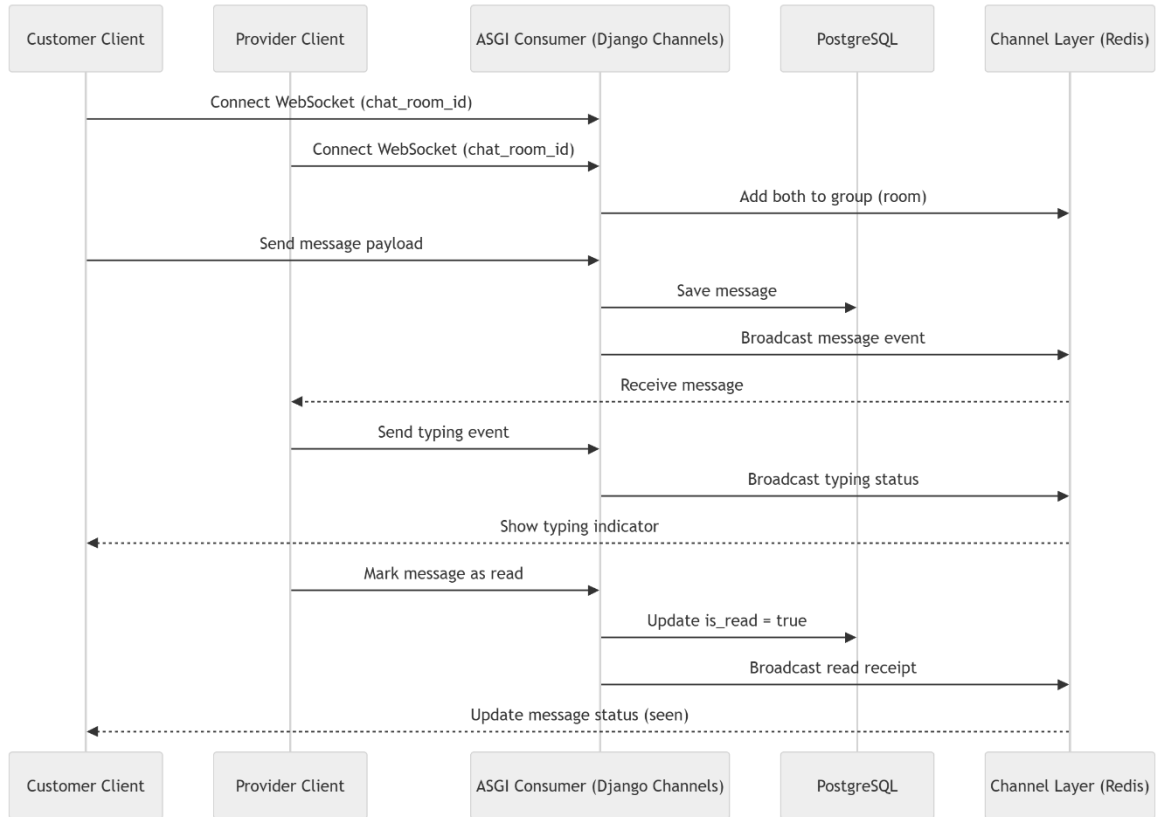
19.2 Marketplace Checkout Flow



Provider Payout Request Flow



Real-Time Chat and Notification Flow



3) App Responsibilities

mainapp

- Primary responsibilities:
- site home and public pages
- custom admin login/dashboard and admin actions
- blog/newsletter
- Stripe platform settings singleton
- activity logs

Key models:

- StripePlatformSettings
- UserActivityLog
- BlogPost
- SignupOTP

customer

Primary responsibilities:

- Signup/login/profile/dashboard
- booking create/manage lifecycle
- Stripe checkout and webhook handling
- notifications center + API
- customer analytics/reviews

Key models:

- CustomerDetails
- Booking
- BookingAddon
- Dispute
- Notification
- MarketplaceOrder / MarketplaceOrderItem
- ProviderPayoutRequest
- StripeWebhookEvent

service_provider

- Primary responsibilities:
- provider signup/login/profile/dashboard
- provider bookings and status updates
- product marketplace (CRUD + approval workflow dependency)
- marketplace cart and checkout
- portfolio gallery
- provider payout request + Stripe Connect helpers

Key models:

- ServiceProviderDetails
- ServiceProviderLogin

- Product
- ServiceAddon
- ProviderPortfolio
- Review
- Message
- PromoCode / ReferralCode / ReferralRedemption

4) URL Topology

Project URL router:

- mysite/mysite/urls.py

Mounted app prefixes:

- /service/ -> service_provider.urls

- /customer/ -> customer.urls

- / -> mainapp.urls

Special admin routing:

- /django-admin/ -> Django admin site

- /admin/ -> custom admin login/dashboard flow

Notable routes:

- Marketplace checkout: /service/marketplace/checkout/
- Booking checkout: /customer/booking/<id>/checkout/
- Stripe webhook: /customer/stripe/webhook/
- Health check: /healthz/

5) Authentication and Session Model

This project uses a mixed auth approach:

- Django auth/session middleware is enabled
- Custom login tables exist:
 - CustomerLogin
 - ServiceProviderLogin

Implication:

- Some flows rely on request.user
- Some flows rely on custom session keys (customer_email/provider_email)
- Custom decorators are used for provider route protection

Recommendation:

- Long-term consolidation into one auth mechanism would reduce complexity and edge cases.

6) Payment, Booking, and Payout Flows

Booking payments (customer app)

- Stripe Checkout session is created in customer checkout flow.
- Success page validates session and updates booking payment state.
- Webhook path exists for robust payment reconciliation and idempotency.

Provider payout pipeline

- Platform/admin receives funds first.
- Commission is computed and persisted per booking/order.
- Eligible completed/paid work can be included in payout requests.
- Admin can process payout request (manual or Stripe transfer).

Marketplace checkout (service_provider app)

Current implemented flow:

1. Customer confirms checkout details.
2. Stripe Checkout session is created and customer is redirected to Stripe.
3. On successful return with session_id, system validates paid status.
4. Marketplace orders and order items are created.
5. Inventory is decremented.
6. Stripe session id is stored on orders for idempotency.

Related recent model additions:

- MarketplaceOrder.stripe_checkout_session_id
- MarketplaceOrder.stripe_payment_intent_id

Migration:

- customer/migrations/0028_marketplaceorder_stripe_fields.py

7) Real-Time Messaging and Notifications

WebSocket stack:

- ASGI with ProtocolTypeRouter in mysite/mysite/asgi.py
- websocket routes in mysite/mysite/routing.py
 - ws/chat/<provider_id>/
 - ws/notifications/

Consumer implementation:

- customer/consumers.py

Capabilities:

- real-time chat messages
- typing indicators
- read receipts
- notification channel support

Channel layer:

- settings currently use InMemoryChannelLayer by default
- Redis configuration is shown as production-ready commented option

Note:

- InMemory layer is not suitable for multi-process production scaling.

8) Data Model Snapshot

Important entities:

- CustomerDetails and ServiceProviderDetails: account profile records
- Booking: service appointment and payment lifecycle
- Product: marketplace listing with stock and fulfillment mode
- MarketplaceOrder: paid order record and provider payout basis
- Notification: customer/provider in-app notices
- ProviderPayoutRequest: payout batching and status

Fulfillment model for marketplace products:

- pickup
- delivery
- both

- digital

9) Background Jobs and Management Commands

Discovered commands:

- customer.management.commands.send_booking_reminders
- mainapp.management.commands.update_account_contacts
- mainapp.management.commands.convert_pkr_to_gbp

Booking reminder command:

- sends reminder emails and creates in-app notifications
- supports dry-run and custom window
- marks Booking.reminder_sent to prevent duplicates

10) Deployment and Runtime

Deployment config files:

- nixpacks.toml
- railway.json
- runtime.txt

Platform assumptions:

- Python 3.11
- Deploy uses Nixpacks
- Start command runs migrate then launches Daphne
- Health endpoint: /healthz/

Current start command pattern:

- cd mysite
- python manage.py migrate
- daphne -b 0.0.0.0 -p \$PORT mysite.asgi:application

11) Configuration Matrix

Key env/config domains:

- Django: SECRET_KEY, DEBUG, ALLOWED_HOSTS, PUBLIC_BASE_URL
- Stripe: publishable/secret/webhook/connect client id
- Storage: Cloudinary credentials or local media
- Email: Brevo API + sender settings
- CSRF trusted origins for localhost + production domains

Stripe configuration precedence:

- DB singleton settings (mainapp.StripePlatformSettings, if active) override settings defaults.

12) Local Development Runbook

Minimal local bootstrap:

1. cd mysite
2. create/activate virtualenv
3. pip install -r requirements.txt
4. python manage.py migrate
5. python manage.py runserver

For Channels production parity:

- run Daphne instead of runserver
- use Redis channel layer

13) Operational Risks and Technical Debt

Security and secrets

- settings.py currently contains a concrete PostgreSQL host/user/password value.
- Recommendation: move all DB credentials to environment variables and rotate exposed credentials.

Auth consistency

- mixed custom auth + Django auth can create session/user mismatches.
- Recommendation: standardize identity resolution and middleware expectations.

Channel layer

- InMemory layer is not horizontally scalable.
- Recommendation: enforce Redis layer in production.

Checkout resilience

- marketplace checkout now supports paid-session idempotency.
- Recommendation: add webhook-based backup reconciliation for marketplace orders similar to booking flow.

14) Testing and QA Checklist

Suggested regression suite:

- customer signup/login and provider signup/login
- booking lifecycle: create -> pay -> complete/cancel/dispute
- Stripe booking payment success and webhook event handling
- marketplace cart CRUD + checkout redirect + paid return

- inventory decrement under concurrent checkout attempts
- payout request creation and admin processing
- chat real-time messaging and read receipts
- notifications API mark-read and mark-all-read

15) File Index (Primary References)

Core:

- mysite/mysite/settings.py
- mysite/mysite/urls.py
- mysite/mysite/asgi.py
- mysite/mysite/routing.py
- mysite/mysite/stripe_config.py

Customer app:

- mysite/customer/models.py
- mysite/customer/views.py
- mysite/customer/urls.py
- mysite/customer/consumers.py
- mysite/customer/management/commands/send_booking_reminders.py

Provider app:

- mysite/service_provider/models.py
- mysite/service_provider/views.py
- mysite/service_provider/urls.py

Main app:

- mysite/mainapp/models.py
- mysite/mainapp/views.py
- mysite/mainapp/urls.py

- nixpacks.toml
- railway.json
- runtime.txt